DESARROLLO WEB · CUADERNO DE PRÁCTICAS

D4 · UD2
Spring Boot

Parámetros HTTP e internacionalización

Kerin Aguilera

JAVA 21 · SPRING BOOT · OCTUBRE DE 2026

Índice

- Objetivo y contenidos
- Procesamiento de datos en HTTP
- Codificación y Query String
- Internacionalización (i18n)
- Conclusiones
- Recursos

Objetivo y contenidos

En esta práctica trabajo con dos aspectos de una aplicación Spring Boot: el envío de datos mediante peticiones HTTP y la selección del idioma de la aplicación.

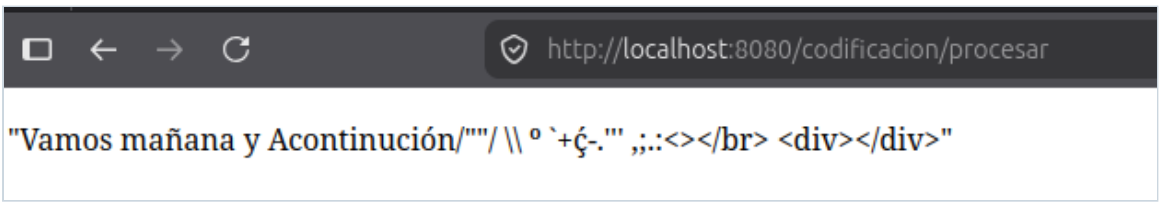
Primero comparo los métodos **POST** y **GET**, reviso un error de mapeo entre el formulario y el controlador, y observo cómo se codifican los parámetros de una URL. Después configuro la internacionalización y compruebo la respuesta en español e inglés.

1. Procesamiento de datos en HTTP

Envío mediante POST

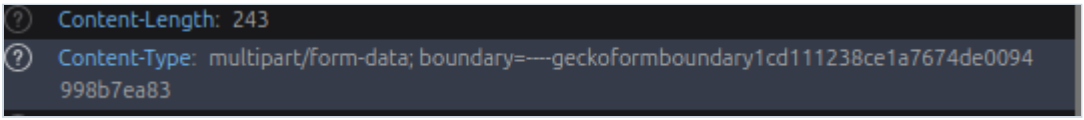
Para comprobar qué datos envía un formulario, utilizo Bruno y reviso la petición recibida. En una petición POST, los datos del formulario se envían en el cuerpo de la petición, no como parte de la dirección URL.

En la primera captura se muestra el envío de un valor:



Petición POST con el valor enviado desde el formulario

También compruebo la cabecera `Content-Type`, que indica el tipo de contenido enviado:



Cabecera Content-Type de la petición

Este es el controlador utilizado para recibir el parámetro `nombre` y añadirlo al modelo:

```
package ruvik.d4ud2.controllador;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/codificacion")
public class controllador {

    @GetMapping
    public String mostrarFormulario() {
        return "formulario";
    }

    @PostMapping("/procesar")
    public String procesar(@RequestParam String nombre, Model modelo) {
        modelo.addAttribute("nombre", nombre);
        return "resultado";
    }
}
```

Un error al enviar una petición GET

Al probar el formulario con el método GET, recibí un error. El controlador tenía estas rutas:

```
@PostMapping("/procesar")
public String procesar(@RequestParam String nombre, Model modelo) {
    modelo.addAttribute("nombre", nombre);
    return "resultado";
}

@GetMapping("/procesar2")
public String procesadoGet(@RequestParam String nombre, Model modelo) {
    modelo.addAttribute("nombre", nombre);
    return "resultado";
}
```

Sin embargo, el formulario enviaba la petición GET a `/codificacion/procesar`:

```
<form method="GET" action="/codificacion/procesar">
```

La ruta `/codificacion/procesar` solo acepta POST; la ruta GET está definida en `/codificacion/procesar2`. Por eso, Spring responde con el error **405 Method Not Allowed**. Para corregirlo, el `action` del formulario debe coincidir con la ruta GET:

```
<form method="GET" action="/codificacion/procesar2">
    <label for="nombre">Nombre:</label>
    <input id="nombre" type="text" name="nombre" required>
    <button type="submit">Enviar</button>
</form>
```

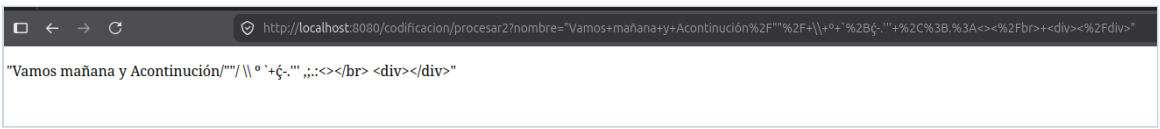
La captura muestra el error antes de aplicar la corrección:



Error 405 al enviar una petición GET a una ruta configurada para POST

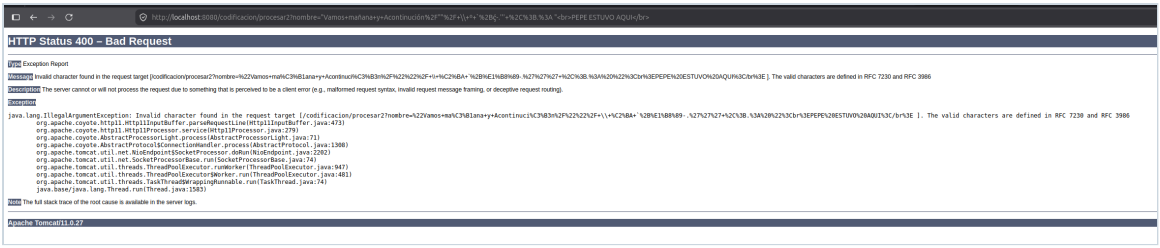
Envío mediante GET

Con GET, los parámetros se añaden a la URL como una cadena de consulta (*query string*). Por eso pueden verse en la barra de direcciones. En la captura se aprecia la URL generada al enviar el formulario:



Parámetro enviado como parte de la URL

También probé a escribir directamente una dirección con caracteres especiales. La petición no se procesó correctamente porque esos caracteres deben codificarse para formar una URL válida:



Error al introducir caracteres especiales sin codificar en la URL

GET resulta práctico para búsquedas y filtros que se puedan compartir mediante un enlace. Como sus parámetros quedan visibles en la URL, no se debe utilizar para enviar contraseñas ni otros datos sensibles.

2. Codificación y Query String

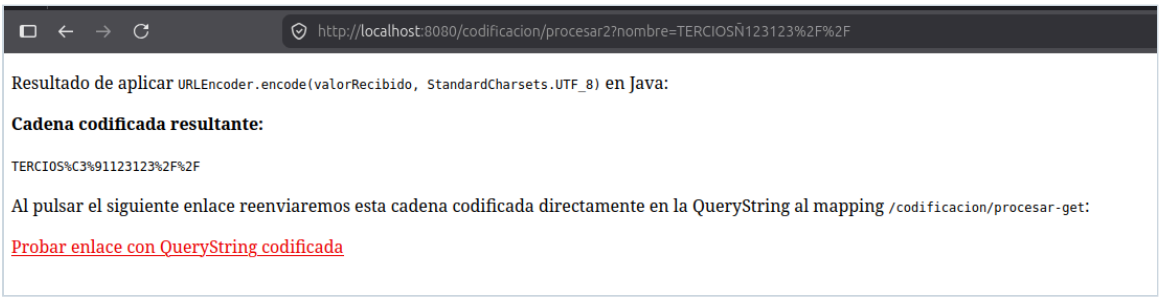
Una *query string* es la parte de la URL que aparece después de `?` y contiene los parámetros de la petición. Cada parámetro se escribe como `nombre=valor`; si hay más de uno, se separan con `&`. Por ejemplo:

```
/codificacion/procesar2?nombre=Ana+Mar%C3%ADa
```

Los espacios, las tildes y otros caracteres reservados deben representarse con una codificación válida para URL. En Java, `URLEncoder.encode(valor, StandardCharsets.UTF_8)` permite codificar un valor antes de incluirlo en una query string. Al recibir la petición, Spring obtiene el valor asociado al parámetro.

Para entender este ejercicio, utilicé una herramienta de inteligencia artificial como apoyo: le pedí que explicara el enunciado y el funcionamiento de la codificación paso a paso. Después apliqué esa explicación para comprender el resultado y completar la práctica.

La captura muestra el valor codificado y el enlace que lo envía en la query string:



Valor codificado con URLEncoder y enlace que lo envía en la query string

3. Internacionalización (i18n)

La internacionalización permite mostrar los textos de la aplicación en distintos idiomas. En esta práctica, el idioma se determina a partir de la petición y los textos se guardan en archivos de mensajes.

El recorrido básico es:

1. La petición proporciona una preferencia de idioma, por ejemplo, mediante la cabecera `Accept-Language` o el parámetro `lang`.
2. Un `LocaleResolver` determina y, según su implementación, mantiene la configuración regional activa.
3. `MessageSource` busca cada clave en el archivo de mensajes correspondiente.

Archivos de mensajes

El archivo de mensajes por defecto, `messages.properties`, contiene los textos en español:

```
app.titulo=Prueba de Internacionalización
app.saludo=¡Hola! Esta aplicación cambia de idioma dinámicamente.
app.instrucciones=Haz clic en los botones para alternar el idioma:
app.idioma.actual=Idioma actual: Español
app.btn.es=Español
app.btn.en=Inglés
```

La variante inglesa, `messages_en.properties`, proporciona los mismos textos en inglés:

```
app.titulo=Internationalization Test
app.saludo=Hello! This application changes language dynamically.
app.instrucciones=Click the buttons below to switch languages:
app.idioma.actual=Current language: English
app.btn.es=Spanish
app.btn.en=English
```

Spring busca las claves de estos archivos de recursos cuando la vista solicita los mensajes.

Alternativa A: cambiar el idioma con `?lang=`

Con `SessionLocaleResolver`, la aplicación puede conservar el idioma en la sesión. `LocaleChangeInterceptor` detecta el parámetro `lang` —por ejemplo, `?lang=en`— y actualiza la configuración regional:

```java
import java.util.Locale;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.LocaleResolver;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.i18n.LocaleChangeInterceptor;
import org.springframework.web.servlet.i18n.SessionLocaleResolver;

@Configuration
public class I18nConfig implements WebMvcConfigurer {

    @Bean
    public LocaleResolver localeResolver() {
        SessionLocaleResolver resolver = new SessionLocaleResolver();
        resolver.setDefaultLocale(Locale.forLanguageTag("es"));
        return resolver;
    }

    @Bean
    public LocaleChangeInterceptor localeChangeInterceptor() {
        LocaleChangeInterceptor interceptor = new LocaleChangeInterceptor();
        interceptor.setParamName("lang");
        return interceptor;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(localeChangeInterceptor());
    }
}
```

El interceptor actúa antes de que el controlador procese la petición: si encuentra el parámetro `lang`, solicita al resolver que actualice el idioma de la sesión.

Alternativa B: usar la cabecera `Accept-Language`

También se puede seleccionar el idioma a partir de la cabecera HTTP `Accept-Language`. Para esta opción configuré `AcceptHeaderLocaleResolver` y limité los idiomas admitidos a español e inglés:

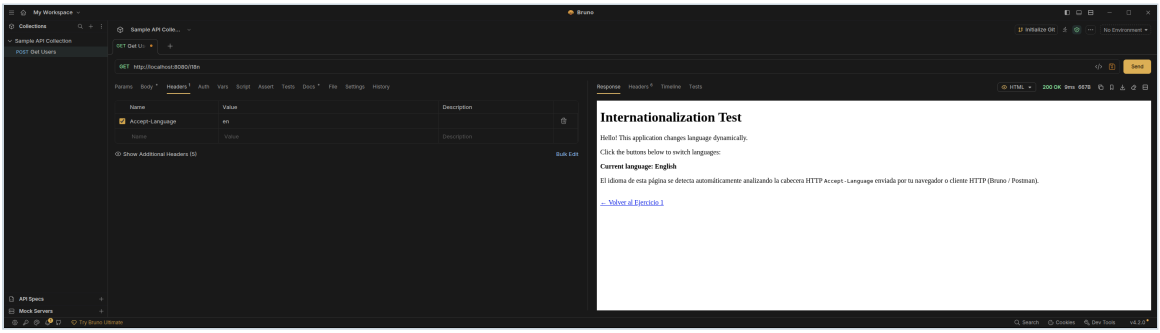
```java
import java.util.Arrays;
import java.util.Locale;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.LocaleResolver;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.i18n.AcceptHeaderLocaleResolver;

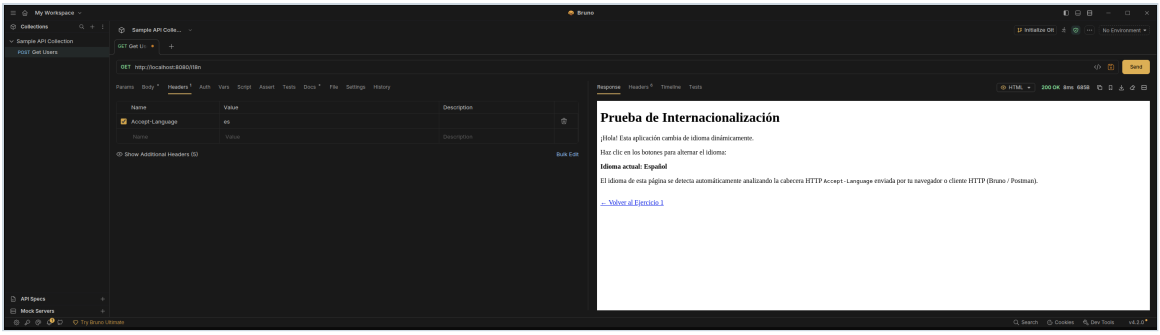
@Configuration
public class I18nConfig implements WebMvcConfigurer {

    @Bean
    public LocaleResolver localeResolver() {
        AcceptHeaderLocaleResolver resolver = new AcceptHeaderLocaleResolver();
        resolver.setDefaultLocale(Locale.forLanguageTag("es"));
        resolver.setSupportedLocales(Arrays.asList(
                Locale.forLanguageTag("es"),
                Locale.forLanguageTag("en")
        ));
        return resolver;
    }
}
```

Estas son **dos alternativas de configuración**: la primera cambia el idioma mediante el parámetro `lang` y una sesión; la segunda lo obtiene de `Accept-Language`. En las capturas comprobé la segunda opción desde Bruno, cambiando el valor de la cabecera:



Respuesta de la aplicación en inglés con Accept-Language: en



Respuesta de la aplicación en español con Accept-Language: es

Conclusiones

La práctica me permitió distinguir dónde viajan los datos en una petición GET y en una POST, y comprobar que el método y la ruta del formulario deben coincidir con el mapeo del controlador. También aprendí que los valores de una URL deben codificarse correctamente y que Spring puede resolver el idioma de una petición mediante una sesión o mediante la cabecera `Accept-Language`.

La parte de internacionalización requirió más investigación: utilicé la IA como apoyo para entender los conceptos y después los relacioné con la configuración y las pruebas realizadas en Bruno.

Recursos

- Repositorio del proyecto en GitHub
- Stack Overflow
- Gemini
- Documentación de las prácticas anteriores